

机器视觉技术实验报告

专业 智能科学与技术 姓名 史峰源 学号 2412526

机器视觉技术课程大作业 Part1——传统手势识别

1 实验目的

本实验面向机器视觉技术课程大作业 Part1，要求在不使用深度学习模型的前提下，完成四类静态手势图像识别任务。识别类别为 A、C、Five 和 V。实验的主要目的如下：

1. 掌握传统图像处理流程中图像读取、前景分割、形态学处理、归一化和特征提取的基本方法。
2. 设计能够描述手势整体形状和局部手指结构的传统视觉特征。
3. 使用传统分类器完成四分类手势识别，并分析分类器参数对识别效果的影响。
4. 对同源数据集进行 K 值交叉验证和完整训练集回测，分析模型复现效果及过拟合风险。
5. 形成可复现的训练程序、测试程序和模型文件，满足课程作业中对传统手势识别程序的要求。

2 实验环境与项目结构

2.1 实验环境

本实验程序使用 C++17 和 OpenCV 实现，使用 CMake 管理工程。当前工程在 Windows 环境下编译运行，主要依赖如下：

- C++17 编译器；
- CMake 3.10 或更高版本；
- OpenCV；
- Visual Studio / MSBuild 编译环境。

2.2 项目结构

整理后的正式工程结构如下：

Listing 1: 项目主要文件结构

```
machine_vision_project_1/
|-- CMakeLists.txt
|-- README.md
|-- part1_traditional_gesture_report.tex
|-- gesture_knn.yml
|-- Hand_Posture_Easy_Stu/
|   |-- A/
|   |-- C/
|   |-- Five/
```

```

|   `-- V/
|-- include/
|   `-- gesture_features.hpp
|-- src/
|   |-- train.cpp
|   `-- test.cpp
|-- test_images/
|   `-- test/
`-- build/
    `-- Release/
        |-- train_gesture.exe
        `-- test_gesture.exe

```

其中，include/gesture_features.hpp 实现手部分割、归一化、特征提取、标准化和加权 KNN 预测；src/train.cpp 为训练程序；src/test.cpp 为测试程序；gesture_knn.yml 为训练得到的模型文件。工程中还保留了若干 debug_masks_* 文件夹，用于调试阶段观察错误样本的归一化 mask，不属于识别主逻辑。

3 实验原理

3.1 总体流程

传统手势识别流程如 算法 1 所示。程序首先从输入图像中分割出手部区域，然后将手部 mask 归一化为固定尺寸，之后提取 HOG、轮廓几何、凸缺陷、手指峰值、投影、网格占空比等特征，最后使用加权 KNN 进行分类。

算法 1 传统手势识别总体流程

Require: 输入手势图像 I ，训练模型 M

Ensure: 手势类别 $y \in \{A, C, Five, V\}$

- 1: 使用 `imdecode` 读取图像
 - 2: 生成多个候选手部 mask
 - 3: 对候选 mask 打分并选择最优 mask
 - 4: 对 mask 做形态学处理、最大连通域提取和填洞
 - 5: 将手部区域裁剪并归一化为 128×128
 - 6: 提取传统视觉特征向量 x
 - 7: 使用训练阶段保存的均值和标准差标准化 x
 - 8: 使用加权 KNN 得到初始类别
 - 9: 根据 Five 和 V 的手指结构特征做少量后处理
 - 10: 输出最终类别
-

3.2 多候选手部分割

手部分割是本实验的关键步骤。由于测试图片可能来自不同背景 and 不同格式，程序没有固定使用单一阈值，而是构造多个候选 mask，并通过打分选择最可信的前景区域。候选来源包括：

1. **透明通道候选：**若输入图像带 alpha 通道，则直接利用透明度提取前景。
2. **边界背景差分候选：**根据图像边界估计背景颜色，计算像素到背景的颜色距离以提取手部。
3. **肤色候选：**结合 HSV 和 YCrCb 颜色空间中的肤色范围得到手部候选区域。

4. **灰度候选**：当手部与背景存在明显亮度差异时，通过灰度阈值得到候选 mask。

候选 mask 的评价指标包括前景面积比例、边界接触比例、外接矩形覆盖范围、extent、solidity、compactness 等。若 mask 面积过小、面积过大、过多贴边或过于接近整块背景，打分会降低。最终选择分数最高的候选作为手部 mask。

3.3 Mask 后处理与归一化

获得初始 mask 后，程序进行二值化、形态学开闭运算、最大连通域提取和填洞。随后根据最大轮廓外接矩形裁剪手部区域，并等比例缩放到 128×128 的标准图像中。该步骤使不同尺寸和不同裁剪比例的手势图像具有统一尺度，便于后续特征提取和分类。

3.4 传统视觉特征

实验中特征维度为 1895，主要由以下几类特征组成：

1. **HOG 特征**：描述手势边缘方向和局部梯度分布。窗口大小为 128×128 ，block 大小为 32×32 ，cell 大小为 16×16 ，方向 bin 数为 9。
2. **轮廓几何特征**：包括面积、周长、外接矩形宽高比、extent、solidity、circularity、凸包面积和凸包周长等，用于描述手势整体形状。
3. **凸缺陷和手指数量特征**：通过凸包和轮廓之间的凸缺陷估计指缝结构，并统计指尖候选数量、上半区垂直投影峰值数量、指缝 valley 数量、指尖展开程度和指缝深度。
4. **V 双指结构特征**：针对 V 手势额外提取两指距离、夹角、两指间 valley 下落深度、两指高度平衡程度和双指匹配分数。
5. **矩、投影和网格特征**：包括质心、偏心率、Hu 不变矩、水平投影、垂直投影以及 8×8 网格占空比，用于描述手势空间分布。

3.5 特征标准化

不同特征的数值尺度差异较大，例如 HOG 特征、面积比例和 Hu 矩的取值范围并不相同。训练阶段对每一维特征计算均值和标准差，测试阶段使用同一组参数进行标准化：

$$x' = \frac{x - \mu}{\sigma} \quad (1)$$

其中， x 为原始特征， μ 为训练集均值， σ 为训练集标准差。标准化后，各维特征在 KNN 距离计算中具有更均衡的影响。

3.6 加权 KNN 分类

本实验最终使用加权 KNN 分类器。对于测试样本，程序计算其到所有训练样本的欧氏距离，取最近的 k 个邻居，并按距离进行加权投票：

$$w_i = \frac{1}{d_i + 10^{-6}} \quad (2)$$

其中， d_i 为测试样本到第 i 个邻居的欧氏距离。距离越近的样本投票权重越大。根据原始数据集拆分验证结果，本实验最终采用 $k = 9$ 。

4 实验步骤

4.1 数据准备

训练数据集为 Hand_Posture_Easy_Stu，共包含 A、C、Five、V 四类手势。每类 50 张图像，共 200 张原始图像。训练阶段对每张图像进行轻微旋转增强，角度为 -10° 、 -5° 、 0° 、 5° 和 10° ，因此最终训练样本数为 1000。

表 1: 训练数据集统计

类别	原始图像数	增强后样本数
A	50	250
C	50	250
Five	50	250
V	50	250
合计	200	1000

4.2 编译工程

工程使用 CMake 编译，命令如下：

Listing 2: 编译命令

```
cmake -S . -B build
cmake --build build --config Release
```

编译完成后生成训练程序和测试程序：

Listing 3: 生成的可执行文件

```
build\Release\train_gesture.exe
build\Release\test_gesture.exe
```

4.3 训练模型

最终使用完整训练集重新训练模型，并固定 KNN 参数 $k = 9$ ：

Listing 4: 训练命令

```
build\Release\train_gesture.exe Hand_Posture_Easy_Stu gesture_knn.yml 9
```

训练完成后生成模型文件 gesture_knn.yml。当前模型文件中的主要字段如下：

Listing 5: 模型文件主要字段

```
model_type: TraditionalGestureWeightedKNN
description: "Part1 traditional vision: segmentation + HOG + contour/finger features + weighted KNN"
k: 9
norm_size: 128
feature_dim: 1895
```

4.4 测试识别

测试程序使用如下命令运行：

Listing 6: 测试命令

```
build\Release\test_gesture.exe test_images\test gesture_knn.yml
```

若需要保存错误样本的归一化 mask，可加入第三个参数：

Listing 7: Debug mask 输出命令

```
build\Release\test_gesture.exe test_images\test gesture_knn.yml debug_masks_current
```

该功能可用于判断错误主要来自分割阶段还是分类阶段。

5 程序代码

本节给出核心程序片段，完整实验源程序见本文末尾“实验源程序清单”，源文件已随工程一同提交。

5.1 训练程序主流程

Listing 8: 训练阶段核心流程

```
cv::Mat trainX;
std::vector<int> trainYVec;

for (int label = 0; label < static_cast<int>(CLASSES.size()); ++label) {
    const std::string& cls = CLASSES[label];
    fs::path clsDir = datasetRoot / cls;
    std::vector<fs::path> files = listPngFiles(clsDir, true);

    for (const auto& p : files) {
        cv::Mat img = imreadAnyPath(p, cv::IMREAD_UNCHANGED);
        if (img.empty()) continue;

        std::vector<cv::Mat> augImgs = makeAugmentations(img);
        for (const cv::Mat& aug : augImgs) {
            cv::Mat feat;
            bool ok = gesture::extractFeature(aug, feat);
            if (!ok || feat.empty()) continue;

            trainX.push_back(feat);
            trainYVec.push_back(label);
        }
    }
}

cv::Mat mean, stddev;
gesture::computeScaler(trainX, mean, stddev);
cv::Mat trainXStd = gesture::standardize(trainX, mean, stddev);
```

5.2 加权 KNN 预测

Listing 9: 加权 KNN 投票核心代码

```
std::vector<double> votes(numClasses, 0.0);

for (int i = 0; i < k; ++i) {
    int label = dist[i].second;
    float d2 = dist[i].first;

    if (label >= 0 && label < numClasses) {
        double d = std::sqrt(std::max(0.0f, d2));
        double w = 1.0 / (d + 1e-6);
        votes[label] += w;
    }
}

int best = nearestLabel;
double bestVote = -1.0;
for (int i = 0; i < numClasses; ++i) {
    if (votes[i] > bestVote) {
        bestVote = votes[i];
        best = i;
    }
}
```

5.3 测试程序主流程

Listing 10: 测试阶段核心流程

```
cv::Mat feat;
cv::Mat debugNormMask;

bool ok = gesture::extractFeature(img, feat, &debugNormMask);
if (!ok || feat.empty()) {
    feat = cv::Mat::zeros(1, mean.cols, CV_32F);
}

cv::Mat featStd = gesture::standardize(feat, mean, stddev);

int pred = gesture::predictWeightedKNN(
    trainX,
    trainLabels,
    featStd,
    k,
    static_cast<int>(classes.size())
);

pred = applyFingerCueFallback(pred, feat, debugNormMask, sourceBorderRatio, classes);
```

```
std::cout << p.filename().string() << " " << classes[pred] << "\n";
```

6 实验结果显示

6.1 K 值选择结果

在原始数据集拆分验证时，测试了多个 KNN 参数。结果如 表 2 所示。

表 2: 不同 K 值的验证结果

k	5 折验证准确率	错误数
1	92.5%	15 / 200
3	94.0%	12 / 200
5	94.0%	12 / 200
7	95.0%	10 / 200
9	95.5%	9 / 200
11	95.5%	9 / 200
15	95.0%	10 / 200
21	95.0%	10 / 200
25	94.5%	11 / 200
31	93.0%	14 / 200

由结果可见， $k = 9$ 和 $k = 11$ 的准确率相同。考虑到 $k = 9$ 更小，更能保留局部邻域差异，因此最终选择 $k = 9$ 。

6.2 完整训练集回测结果

使用完整 Hand_Posture_Easy_Stu 训练后，对同一数据集进行回测，结果如 表 3 所示。

表 3: 完整训练集回测结果

类别	样本数	错误数	正确率
A	50	0	100%
C	50	0	100%
Five	50	0	100%
V	50	0	100%
合计	200	0	100%

该结果主要用于验证训练产物、特征维度和测试程序一致。由于训练集和测试集相同，该结果不能代表独立泛化能力。结合 5 折交叉验证结果看， $k = 9$ 的验证准确率为 95.5%，低于完整训练集回测的 100%，但差距不大，说明模型存在一定记忆训练样本的可能，不过在课程同源测试场景下过拟合风险可接受。

7 实验分析总结

7.1 结果分析

本实验最终模型在同源训练数据上能够稳定识别四类手势。对原数据集拆分验证时， $k = 9$ 的整体表现较好，因此最终使用 $k = 9$ 在完整训练集上重新训练。模型文件中保存了标准化参数和全部训练特征，测试阶段无需重新训练即可直接识别。

从过拟合角度看，完整训练集回测达到 100%，只能说明训练产物、特征提取流程和测试程序能够正确匹配，不能单独证明模型具有完全泛化能力。相比之下，5 折交叉验证中 $k = 9$ 的准确率为 95.5%，与训练集回测存在一定差距但并不异常。由于课程最终测试与训练数据同源，当前模型复杂度和 k 值选择基本合理；若测试集改为跨域图片，传统特征和 KNN 仍可能出现准确率下降。

7.2 方法优点

1. 方法完全基于传统视觉，不依赖深度学习训练环境，符合 Part1 要求。
2. 特征具有较强可解释性，可以通过 mask、指尖数量、凸缺陷数量等中间结果分析错误原因。
3. 使用多候选分割策略，提高了对不同背景 and 不同图片格式的适应能力。
4. 针对 Five 和 V 增加了手指数量和双指角度特征，使模型对张开手指类手势更敏感。

7.3 局限性

1. 传统特征依赖分割质量，复杂背景和强光照变化仍可能导致 mask 异常。
2. 若未来测试集与课程数据集采集规则差异较大，裁剪方式、手势习惯和背景变化仍可能影响准确率。
3. A 与 V 在部分非标准姿态下轮廓相似，单纯依靠轮廓和指尖规则仍可能混淆。
4. 后处理规则具有一定经验性，适合当前四分类任务，不适合直接扩展到大量手势类别。

7.4 结论

本实验完成了一个传统视觉手势识别系统。系统通过多候选分割获得手部 mask，经过归一化后提取 HOG、轮廓几何、凸缺陷、手指数量、V 双指结构、投影和网格占空比等特征，并使用加权 KNN 完成分类。实验表明，该方法在同源数据集上表现稳定。结合训练集回测和 5 折交叉验证结果，当前方案在课程作业的同源测试场景下具有较好的可复现性和可解释性。

8 实验源程序清单

完整实验源程序已随本报告放在同一工程目录中，主要文件如下：

1. CMakeLists.txt: CMake 工程配置文件。
2. include/gesture_features.hpp: 手部分割、特征提取、标准化和 KNN 预测。
3. src/train.cpp: 模型训练程序。
4. src/test.cpp: 模型测试程序。
5. gesture_knn.yml: 使用完整训练集和 $k = 9$ 训练得到的模型文件。
6. README.md: 工程说明文档。